

Project Proposal – Knowledge-Based Systems

Vehicle Fault Diagnostics System

A system that infers probable vehicle faults based on driver-reported symptoms (sounds, vibrations, dashboard warnings) and OBD-II data, prioritizes them by severity, and recommends appropriate repairs.

1. Team Members

Member 1

Vladimir Morozkin | SV85/2023

Module 1 – Symptom & OBD-II diagnosis

Member 2

Danila Travkov | SV86/2023

Module 2 – Fault prioritization & repair recommendation

The knowledge model, domain classes, and shared rule infrastructure are designed jointly to ensure consistency of core concepts (Vehicle, Symptom, DTCCode, SensorReading, FaultCandidate, Diagnosis) across both modules.

2. Problem Description

2.1 Motivation

Modern automobiles contain dozens of electronic control units (ECUs) and hundreds of sensors. When something goes wrong, the average driver faces two immediate problems: they do not know whether the fault is dangerous and requires stopping immediately, and they do not know which type of specialist to contact. Misreading the situation can lead to engine seizure, a traffic accident, or an unnecessarily expensive repair caused by delayed action.

Rule-based expert systems are a natural fit here. They reason over a clearly defined knowledge base and can always explain *why* a particular conclusion was reached — a property that is especially valuable when the stakes involve personal safety and financial decisions.

2.2 Problem Overview and Existing Solutions

The system targets **automated vehicle fault diagnosis**: given a structured set of driver-reported symptoms and standardized OBD-II diagnostic data (DTC error codes, live sensor values), produce a ranked list of probable faults with confidence scores, urgency ratings, and actionable repair recommendations.

Review of existing solutions:

- **OBD-II readers (ELM327, FIXD, BlueDriver):** Read DTC codes and display short descriptions. They perform no multi-level inference, do not correlate symptoms with sensor trends, and do not assess urgency or causal relationships between codes.
- **Professional diagnostic stations (Bosch ESI, Delphi DS):** Designed for trained mechanics with expensive equipment. Not accessible to ordinary drivers.
- **Mobile applications (Car Scanner, Torque Pro):** Visualize raw OBD-II data but act purely as display tools — no expert reasoning layer.
- **Academic research:** Papers on expert systems in automotive diagnostics exist (e.g., IDES – Intelligent Diesel Engine System, Delphi et al., 2010), but they target specific engine types and are not available as general solutions.

Advantages of the proposed solution:

- Multi-level forward-chaining that combines symptoms, DTC codes, and sensor values into a diagnosis — absent from any OBD reader on the market.
- A CEP module that detects dangerous trends in sensor time-series (overheating trajectory, voltage oscillations, sporadic misfire patterns) *before* the fault manifests as a visible symptom.
- Backward chaining for hypothesis verification — "Is the root cause X?"
- Fault prioritization by urgency: CRITICAL / HIGH / MEDIUM / LOW, with concrete repair guidance.
- Adaptability to different vehicle types via a template mechanism that parameterizes sensor thresholds by engine type and vehicle model year.

3. Methodology

3.1 System Inputs

3.1.1 Driver-reported symptoms

Category	Examples
Sounds	Knocking on acceleration, brake squeal, droning at speed, whining on startup, hissing
Vibrations	Steering wheel vibration under braking, idle vibration, speed-dependent shaking
Dashboard warnings	Check Engine, ABS, Airbag, Oil Pressure, Coolant Temp, TPMS, Battery
Subjective observations	Power loss, increased fuel consumption, hard starting, exhaust smoke (white/black/blue)

Category	Examples
Context of occurrence	Always / only when braking / only on acceleration / only cold / after extended driving

3.1.2 OBD-II diagnostic data

Data type	Details
DTC codes	P (powertrain), B (body), C (chassis), U (network) — active, pending, or historic
Live sensor values	Coolant temp, battery voltage, fuel pressure, O2 sensor, MAF, MAP, RPM, vehicle speed, IAT
Freeze frame	Sensor snapshot at the moment a DTC was set
Readiness monitors	Ready / not ready status per emission subsystem
Vehicle metadata	VIN, engine type (petrol/diesel/hybrid), model year, odometer

3.2 System Outputs

- **Diagnosis list:** Ranked probable faults with confidence scores (e.g., "Faulty lambda sensor – 87%", "Clogged EGR valve – 64%")
- **Urgency rating per fault:** CRITICAL (vehicle must not be driven), HIGH (repair within 48h), MEDIUM (repair before next service), LOW (monitor, not urgent)
- **Recommended actions:** Service type (auto-electrician, mechanic, tire shop), estimated cost range, what *not* to do until repaired
- **Rule trace:** The complete chain of fired rules that led to each conclusion
- **Secondary fault warnings:** Faults that may develop if the primary fault is not addressed

3.3 Knowledge Base

The knowledge base is organized into three layers corresponding to the three levels of forward-chaining inference. It is populated from official technical standards (SAE J1979, ISO 15031), manufacturer diagnostic manuals (ALLDATA, Mitchell1), and expert knowledge gathered from experienced automotive mechanics.

Layer	Content
Layer 1 – Primary facts	Instances of: Vehicle, Symptom, DTCCode, SensorReading, OccurrenceContext

Layer	Content
Layer 2 – Derived diagnostic facts	Affected system, individual symptom severity, DTC classification, correlated symptom groups
Layer 3 – Diagnosis & recommendation	Final fault diagnosis, urgency classification, repair recommendation, rule trace

4. Example Rules in Human-Readable Form

Before presenting the full Drools implementation, the table below illustrates the basic reasoning logic of the system using plain IF-THEN notation. Each rule covers a different vehicle system and a different urgency level.

#	System	IF (conditions)	THEN (conclusion)	Urgency
1	Engine / Lubrication	Oil pressure warning is active AND oil pressure sensor reads below minimum AND engine is running	Probable cause: oil pump failure or severe oil loss. Stop the engine immediately. Flag as critical fault.	CRITICAL
2	Cooling	Coolant temperature sensor exceeds maximum threshold AND no coolant leak is detected AND heater output is cold	Probable cause: faulty thermostat (stuck closed). Schedule urgent repair. Do not drive at highway speeds.	HIGH
3	Ignition / Fuel	DTC code P0301–P0304 is active (cylinder-specific misfire) AND driver reports power loss AND fuel trim is within normal range	Probable cause: worn spark plug or faulty ignition coil on the affected cylinder. Repair before next service interval.	MEDIUM
4	Tyres / TPMS	TPMS warning is active AND reported tyre pressure is 10–20% below the recommended value AND no sudden pressure drop detected	Probable cause: slow puncture or natural pressure loss. Inflate tyres and monitor. No immediate action required.	LOW

These four rules already demonstrate the core structure of the knowledge base: conditions drawn from sensor readings, DTC codes, and driver-reported symptoms are combined to produce a diagnosis with an urgency rating. The full rule base extends this pattern across all major vehicle systems with multi-level chaining, CEP-based trend detection, and template-driven threshold parameterization.

5. Rule Base

5.1 Forward Chaining – Symptom Diagnosis (Module 1)

Three levels of chaining are required for a grade of 8 or above. All three are implemented here.

Level 1 – Identify affected system:

```
rule "Cooling system problem – sensor threshold exceeded"
when
    Symptom(type == WARNING_COOLANT_TEMP)
    or SensorReading(name == "coolant_temp", value >
vehicleThresholds.maxCoolantTemp)
then
    insert(new SystemProblem(COOLING_SYSTEM, ACTIVE));
end

rule "Ignition system problem – misfire DTC"
when
    DTCCode(code matches "P030[0-9]")    // P0300–P0309: random/cylinder-
specific misfire
then
    insert(new SystemProblem(IGNITION_SYSTEM, ACTIVE));
end
```

Level 2 – Identify faulty component:

```
rule "Faulty thermostat – overheating without leak or smoke"
when
    SystemProblem(type == COOLING_SYSTEM)
    not Symptom(type == FLUID_LEAK)
    not Symptom(type == EXHAUST_SMOKE)
    SensorReading(name == "coolant_temp", value > 105)
then
    insert(new FaultCandidate(THERMOSTAT, 0.78, HIGH));
end

rule "Faulty water pump – overheating with grinding noise"
when
    SystemProblem(type == COOLING_SYSTEM)
    Symptom(type == GRINDING_SOUND, context == WHILE_ENGINE_RUNNING)
then
    insert(new FaultCandidate(WATER_PUMP, 0.71, HIGH));
end
```

Level 3 – Final diagnosis and prioritization:

```
rule "Final diagnosis – high-confidence candidate"
when
    $k : FaultCandidate($comp : component, confidence >= 0.75, $urg :
urgency)
    not FaultCandidate(component == $comp, confidence > $k.confidence)
then
    insert(new Diagnosis($comp, $k.confidence, $urg,
        generateRecommendation($comp), getRuleTrace()));
end

rule "Escalate to CRITICAL – oil pressure + overheating"
when
    FaultCandidate(urgency == HIGH)
    (Symptom(type == EXHAUST_SMOKE, color == WHITE)
    or SensorReading(name == "oil_pressure",
        value < vehicleThresholds.minOilPressure))
then
    insert(new UrgencyFlag(CRITICAL, "Risk of engine seizure"));
end
```

5.2 CEP – Sensor Time-Series Trend Detection

OBD-II sensors report values every few seconds. The CEP module aggregates these streams and detects dangerous trends *before* a fault becomes a visible symptom.

```
// Coolant temperature rising trend
rule "Overheating trend – sustained temperature rise over 5 minutes"
when
    $start : SensorReading(name == "coolant_temp", $t0 : value)
        over window:time(5m) from entry-point "obdStream"
    Number(doubleValue > $t0 + 15) from accumulate(
        SensorReading(name == "coolant_temp", value > $t0)
        over window:time(5m) from entry-point "obdStream",
        max(value))
then
    insert(new CEPEvent(OVERHEATING_TREND, getCurrentTime()));
end

// Unstable battery voltage – sign of alternator or battery failure
rule "Battery voltage oscillation over 2 minutes"
when
    Number(doubleValue > 1.5) from accumulate(
        SensorReading(name == "battery_voltage", $v : value)
```

```

        over window:time(2m) from entry-point "obdStream",
        (max(value) - min(value)))
    then
        insert(new SystemProblem(ELECTRICAL_SYSTEM, UNSTABLE));
    end

// Sporadic misfire pattern – points to ignition or fueling issues
rule "Sporadic misfire – 3 or more occurrences in 10 minutes"
when
    Number(intValue >= 3) from accumulate(
        DTCCode(code matches "P030[0-9]") over window:time(10m)
        from entry-point "dtcStream",
        count(1))
then
    insert(new SystemProblem(IGNITION_SYSTEM, SPORADIC_MISFIRE));
end

```

5.3 Template Mechanism – Parameterization by Vehicle Type

Critical sensor thresholds differ between engine types (petrol / diesel / hybrid) and model years. Instead of writing separate rules for each configuration, a `.drt` template generates concrete DRL rules from a parameter table at runtime. Changing a threshold (e.g., after a regulation update) requires only updating the table — no DRL code changes.

```

// template: sensor_thresholds.drt
template header
engineType  maxCoolantTemp  minOilPressure  voltageMin  voltageMax
maxMafNormal
end

template "Coolant overheating check – @{engineType}"
rule "Coolant overheating – @{engineType}"
when
    Vehicle(engineType == "@{engineType}")
    SensorReading(name == "coolant_temp",
        value > @{maxCoolantTemp})
then
    insert(new SystemProblem(COOLING_SYSTEM, CRITICAL));
end
end template

// Parameter table (CSV):
// engineType | maxCoolantTemp | minOilPressure | voltageMin | voltageMax |
// maxMafNormal
// PETROL    |      110      |      1.0      |      13.5  |      14.8  |

```

```

22
// DIESEL      |      105      |      1.2      |      13.5      |      14.8      |
22
// HYBRID      |      100      |      0.9      |      13.8      |      15.2      |
18

```

5.4 Backward Chaining – Fault Hypothesis Verification (Module 2)

Module 2 allows the system to verify a directly stated hypothesis ("Is the fault in the lambda sensor?") by recursively checking whether all required conditions are satisfied in the working memory, rather than always reasoning bottom-up from symptoms.

```

// Backward chaining query – lambda sensor fault hypothesis
query "fault_lambda" (String vehicleId)
    DTCCode(code == "P0131" || code == "P0132" || code == "P0134")
    or (SensorReading(name == "o2_sensor_b1s1",
        value < 0.1 || value > 0.9) // no oscillation = dead sensor
        and Symptom(type == INCREASED_FUEL_CONSUMPTION))
end

// Recursive causal chain for multi-vehicle fault propagation
query "fault_root_cause" (Component $root)
    $root := directCause(ActiveFault)
    or (directCause($intermediate)
        and fault_root_cause($intermediate, $root))
end

```

Fault prioritization with accumulate:

```

rule "Accumulate urgency – safety-critical faults"
when
    $list : List() from accumulate(
        Diagnosis($urg : urgency, $comp : component,
            safetyRisk == true),
        collectList($comp))
    eval($list.size() >= 1)
then
    insert(new Recommendation(STOP_DRIVING_IMMEDIATELY,
        "Safety-critical fault detected: " + $list.toString(), $list));
end

rule "Combined fault escalation – cooling + electrical"
when
    $d1 : Diagnosis(urgency == HIGH, system == COOLING_SYSTEM)

```



```
Diagnosis(urgency == HIGH, system == ELECTRICAL_SYSTEM)
then
    modify($d1) { setUrgency(CRITICAL); }
end
```

6. Step-by-Step Reasoning Example

6.1 Input Data

- **Vehicle:** Volkswagen Golf 5, 2.0 TDI (diesel), 2007, 187,000 km
- **Driver-reported symptoms:** Check Engine light on, black exhaust smoke under acceleration, noticeable power loss
- **OBD-II DTC codes:** P0401 (EGR Flow Insufficient Detected), P0234 (Turbocharger Overpressure)
- **Live sensor values:** MAF = 8.2 g/s (expected 12–18 for this engine), boost pressure = 2.3 bar (max 2.0)

6.2 Reasoning Steps

Step 1 – Template parameterization for diesel engine:

The parameter table entry for DIESEL generates rules with maxCoolantTemp = 105, normalMAF = 10–22 g/s. The current MAF of 8.2 g/s falls below the normal range -> `insert(LowSensorValue(MAF))`.

Step 2 – Level 1: Identify affected systems:

- Rule for P0401 fires -> `insert(SystemProblem(EGR_SYSTEM, ACTIVE))`
- Rule for P0234 fires -> `insert(SystemProblem(TURBO_SYSTEM, ACTIVE))`
- Black smoke symptom -> `insert(SystemProblem(FUEL_SYSTEM, SUSPECTED))`

Step 3 – CEP trend detection:

Over the last 5 minutes, MAF readings show a continuous decline: 14.2 -> 11.8 -> 9.6 -> 8.2 g/s. The CEP trend rule fires -> `insert(CEPEvent(MAF_DECLINING_TREND, timestamp))`.

Step 4 – Level 2: Identify faulty component:

- EGR_SYSTEM + MAF_DECLINING_TREND + DTC P0401 -> `insert(FaultCandidate(EGR_VALVE, 0.84, HIGH))`
- TURBO_SYSTEM + P0234 + boost pressure above max -> `insert(FaultCandidate(TURBO_ACTUATOR, 0.71, HIGH))`

Step 5 – Backward chaining hypothesis verification:

The system poses the hypothesis "EGR valve fault" and the query checks recursively: P0401

present? YES. MAF below range? YES. Intake restriction confirmed? YES (black smoke). Hypothesis confirmed at 84% confidence.

Step 6 – Level 3: Final diagnosis and recommendation:

Accumulate combines the candidates. The Level 3 rule fires and generates:

- **Primary fault:** Clogged / faulty EGR valve (84% confidence)
- **Secondary fault:** Turbo actuator (71% confidence)
- **Urgency:** HIGH
- **Recommendation:** Diesel specialist service, EGR valve cleaning or replacement, turbo system inspection. Do not drive at motorway speeds.

Fired rule trace:

```
TemplateRule_DIESEL -> IdentifyEGR_P0401 -> IdentifyTurbo_P0234 ->
CEP_MAFDecliningTrend -> CandidateEGR -> CandidateTurbo -> BackwardChaining_EGR
-> CombinedFaultEscalation -> FinalDiagnosis_EGR -> FinalRecommendation
```

6.3 What This Example Demonstrates

Technique	Where it appears
Forward chaining – Level 1	System identification from DTC codes and symptoms
Forward chaining – Level 2	Component candidate inference from correlated facts
Forward chaining – Level 3	Final diagnosis + recommendation from accumulated candidates
CEP	Detection of MAF declining trend in 5-minute time window
Template	Diesel-specific sensor thresholds generated from parameter table
Backward chaining	EGR valve hypothesis recursively verified in working memory
Accumulate	Weighted candidate scoring (confidence) and urgency summation

7. Technique Distribution Between Team Members

Technique	Member 1 – Module 1	Member 2 – Module 2
Forward chaining (3+ levels)	(symptoms -> fault candidate)	(candidates -> priority + recommendation)

Technique	Member 1 – Module 1	Member 2 – Module 2
CEP	(sensor trends, misfire pattern, MAF decline)	(voltage oscillation, tyre pressure drop, repeated warnings)
Templates (.drt)	(thresholds by engine type)	—
Backward chaining	—	(fault hypothesis verification)
Accumulate	(aggregated sensor readings)	(urgency score summation)

Both members implement forward chaining with a minimum of three chaining levels and use the `accumulate` function. CEP, templates, and backward chaining are each covered by at least one team member, satisfying the requirements for a grade of 9–10.

8. Conclusion

The proposed vehicle fault diagnostics system delivers multi-layer expert reasoning that combines driver-reported symptoms, OBD-II data, and sensor time-series trends into actionable fault diagnoses. It is designed to be understandable to a non-technical driver while providing transparent, traceable reasoning for every conclusion. By implementing all three advanced techniques required by the course (CEP, templates, backward chaining) alongside forward chaining with three or more levels of inference, the system reaches a diagnostic capability that substantially exceeds existing commercial OBD readers.

9. Rule Base and Class Diagrams

9.1 Basic Rules

Member 1 - Module 1: Symptom and OBD-II Diagnosis

ID	Area	IF	THEN
M1-BR-01	Engine / DTC	IF Check Engine is active AND DTC P0300 is present	THEN create random misfire symptom group and mark Ignition System as suspect.
M1-BR-02	Ignition	IF DTC P0301-P0304 is present AND power loss is reported	THEN create cylinder-specific misfire candidate with MEDIUM urgency.
M1-BR-03	Cooling	IF coolant temperature is above threshold AND coolant warning is active	THEN create Cooling System problem with HIGH urgency.
M1-BR-04	Lubrication	IF oil pressure warning is active AND oil pressure value is below minimum	THEN create Oil Pressure fault with CRITICAL urgency.
M1-BR-05	Fuel / Air	IF black exhaust smoke is reported AND MAF value is below expected range	THEN mark air intake restriction or EGR flow problem as suspected.
M1-BR-06	Exhaust / O2	IF DTC P0131 or P0132 is present AND fuel consumption increased	THEN create lambda sensor fault candidate.
M1-BR-07	Electrical	IF battery warning is active AND voltage is below 12.0V while engine is running	THEN create charging system problem candidate.
M1-BR-08	Braking	IF brake squeal is reported AND vibration occurs while braking	THEN create brake pad or warped disc candidate with HIGH urgency.
M1-BR-09	Tyres / TPMS	IF TPMS warning is active AND tyre pressure is 10-20% below recommended	THEN create low tyre pressure candidate with LOW urgency.
M1-BR-10	Transmission	IF jerking occurs during gear change AND no engine misfire DTC is present	THEN create transmission or clutch diagnostic candidate.

Member 2 - Module 2: Fault Prioritization and Repair Recommendation

ID	Area	IF	THEN
M2-BR-01	Prioritization	IF a diagnosis has CRITICAL urgency	THEN recommend STOP_DRIVING_IMMEDIATELY and place it first in the ranked list.
M2-BR-02	Prioritization	IF two HIGH urgency diagnoses affect the same vehicle system	THEN escalate combined risk level to CRITICAL.
M2-BR-03	Repair routing	IF fault system is ELECTRICAL_SYSTEM	THEN recommend auto-electrician service.
M2-BR-04	Repair routing	IF fault system is BRAKE_SYSTEM	THEN recommend immediate mechanic / brake service inspection.
M2-BR-05	Repair routing	IF fault system is TYRE_SYSTEM AND pressure loss is slow	THEN recommend tyre shop inspection and pressure monitoring.
M2-BR-06	Cost estimate	IF candidate component is lambda sensor	THEN attach estimated repair cost range LOW_TO_MEDIUM.
M2-BR-07	Cost estimate	IF candidate component is turbo actuator	THEN attach estimated repair cost range HIGH.
M2-BR-08	Driver warning	IF exhaust smoke is white AND overheating is detected	THEN warn about head gasket or coolant leak risk.
M2-BR-09	Secondary fault	IF EGR fault remains active AND black smoke is present	THEN add secondary warning for DPF clogging risk.
M2-BR-10	Recommendation	IF confidence is below 0.60 AND urgency is not CRITICAL	THEN recommend monitoring and additional diagnostic scan before repair.

9.2 CEP Rules

Member 1 CEP Rules

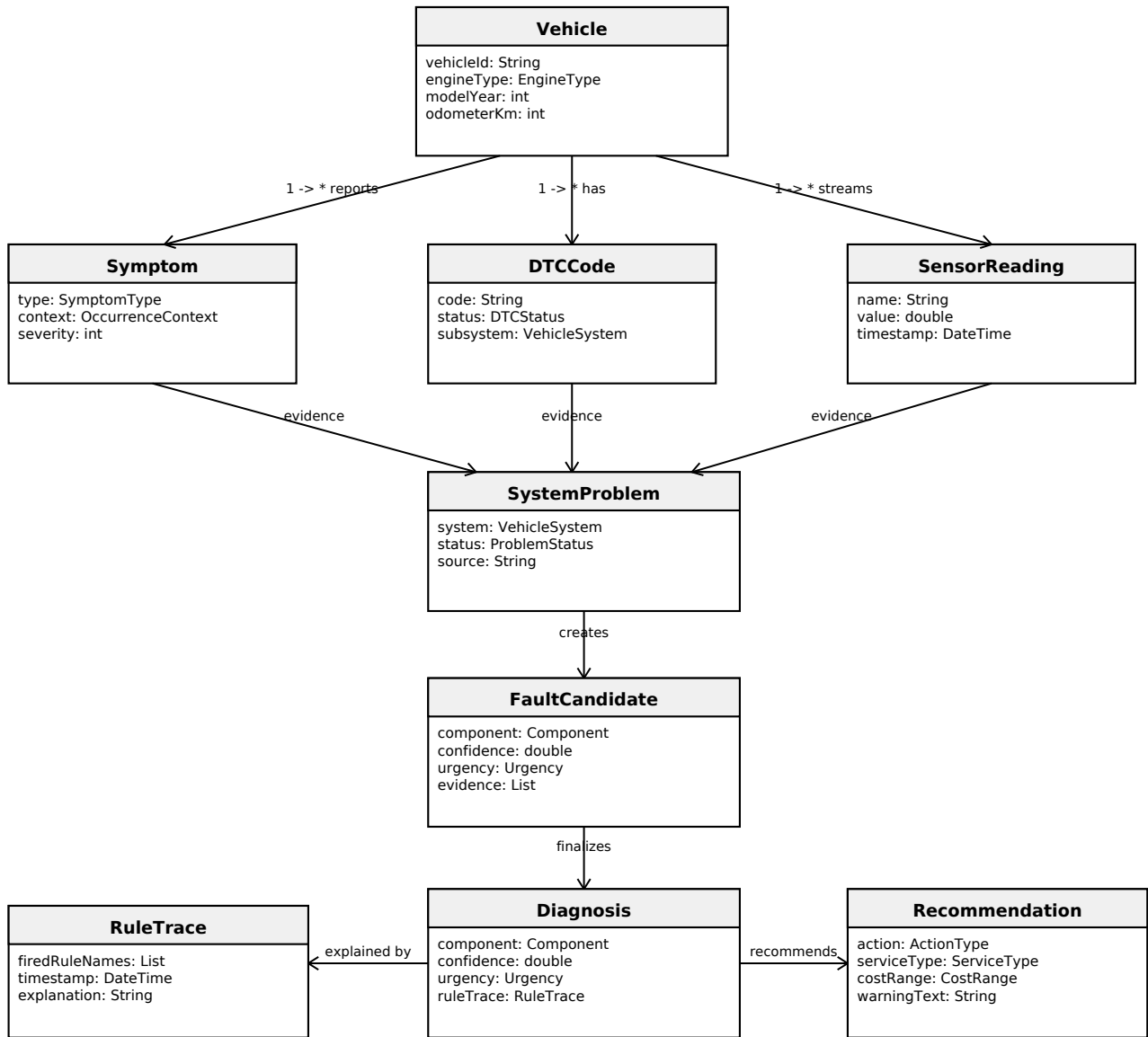
ID	CEP rule	Event pattern	Result
M1-CEP-01	Coolant overheating trend	SensorReading(coolant_temp) over 5m, max(value) > start + 15	Insert CEPEvent(OVERHEATING_TREND).
M1-CEP-02	Sporadic misfire pattern	At least 3 P030x DTC events in 10m	Insert SystemProblem(IGNITION_SYSTEM, SPORADIC_MISFIRE).
M1-CEP-03	MAF declining trend	MAF readings continuously decline over 5m and final value below threshold	Insert CEPEvent(MAF_DECLINING_TREND).

Member 2 CEP Rules

ID	CEP rule	Event pattern	Result
M2-CEP-01	Battery voltage oscillation	Voltage min/max difference > 1.5V over 2m	Escalate electrical diagnosis and recommend charging system inspection.
M2-CEP-02	Rapid tyre pressure drop	TPMS pressure drops > 15% in 3m	Escalate tyre fault to HIGH and warn driver to stop safely.
M2-CEP-03	Repeated critical warnings	Two or more CRITICAL warning events within 10m	Create global STOP_DRIVING recommendation and mark top priority.

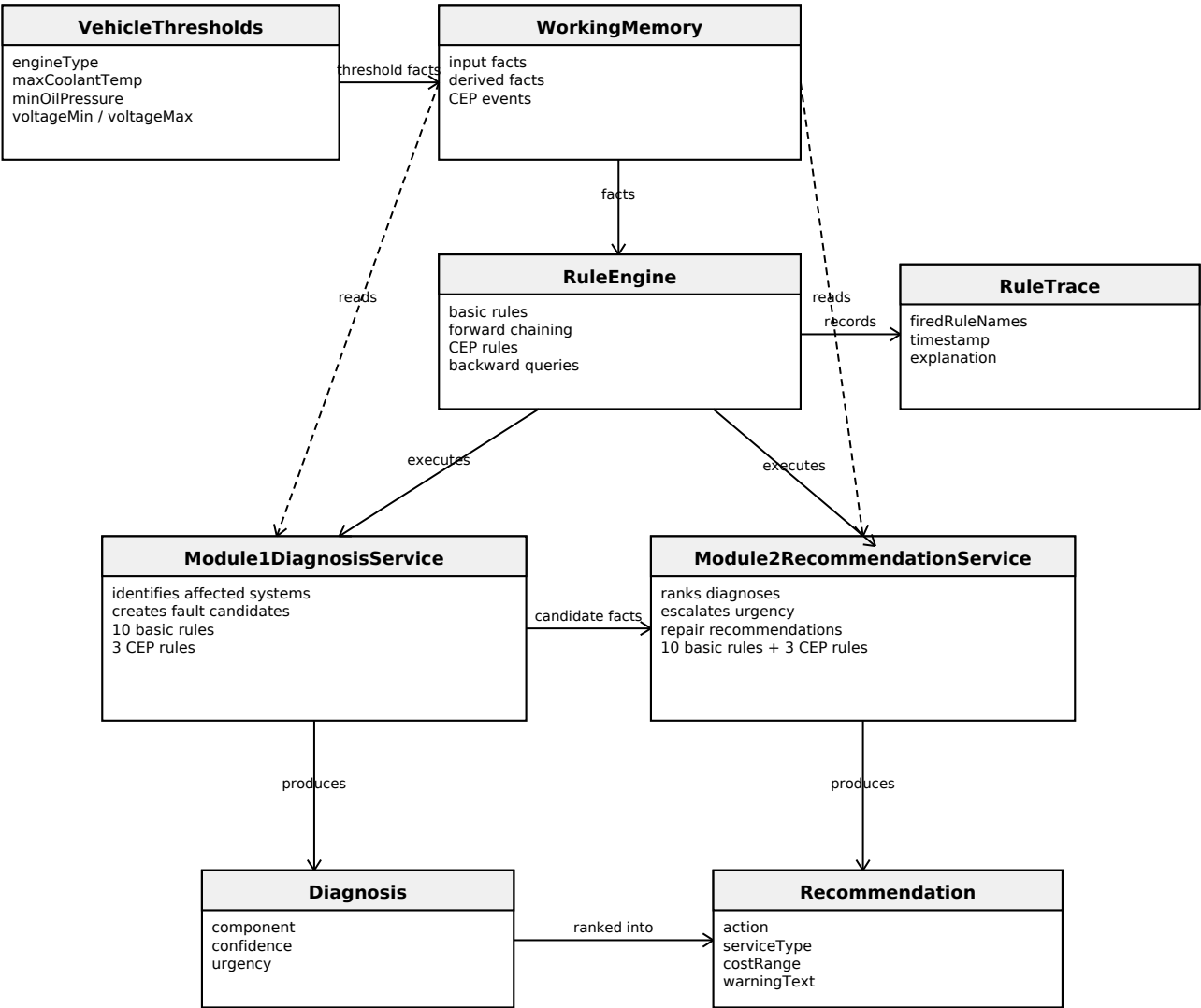
9.3 Class Diagrams

9.3.1 Domain Class Diagram



Domain model: facts from Vehicle, Symptom, DTCCode, and SensorReading are inferred into SystemProblem, FaultCandidate, Diagnosis, and Recommendation.

9.3.2 Rule Infrastructure and Module Responsibility Diagram



Module responsibility view: Module 1 derives diagnostic candidates; Module 2 prioritizes them and creates repair guidance.